Informe — Parcial práctico de Ingeniería de Software: «La última transmisión»

> Borrador. Máximo 8 páginas de cuerpo al exportar a PDF. Los diagramas, la matriz completa y las capturas del tablero están en el **Anexo** (`anexo.pdf`); aquí solo se referencian.

1. Identificación

Autor	[COMPLETAR: nombre completo]
Modalidad	Individual; la guía prevé parejas. [COMPLETAR: motivo y si el docente lo autorizó (medio y fecha).]
Curso / grupo	[COMPLETAR]
Docente	[COMPLETAR]
Apertura y cierre del parcial	[COMPLETAR: fecha y horas]
Repositorio / versiones	Git local; etiquetas `v1-inicial` (antes del cambio) y `v2-cambio` (después)
Video	[COMPLETAR: `entrega/video.mp4` o enlace con acceso para el docente]

2. Problema, actores, alcance y calidad

Problema. Un colectivo de escritores necesita un motor que interprete narraciones interactivas escritas en archivos, muestre solo las decisiones compatibles con lo ocurrido, aplique sus consecuencias y detecte errores del contenido antes de que un lector lo juegue.

Actores. *Escritora principal* (edita archivos sin programar), *diseñador de la experiencia* (decisiones con consecuencias y opciones condicionadas), *coordinadora del festival* (errores comprensibles, sin cierres inesperados) y *lector* (juega en consola).

Alcance. Incluye: carga de historias JSON, validación completa con 10 códigos de error, sesión con condiciones de igualdad booleanas y efectos de asignación, finales, bloqueo, reinicio y —tras el cambio— deshacer un nivel; interfaz de consola local. Excluye (según la guía): cuentas, base de datos, guardado entre ejecuciones, red, editor visual, multimedia y lenguaje de expresiones.

Riesgos y mitigación.

Riesgo	Mitigación aplicada
R1. Un error en el contenido provoca un cierre inesperado durante la partida.	Validación completa antes de jugar (AD-1), captura general en `cli.main`, RNF-02 y RNF-03 verificados (0 tracebacks, 0 excepciones en 100 000 acciones).
R2. Lógica específica de la historia oficial dentro del motor impide cargar otras historias.	Contrato de contenido documentado, segunda historia `el_faro.json`, prueba que busca IDs/variables de las historias en `motor/` (RNF-01).

Priorización de atributos de calidad. (1) **Confiabilidad/corrección de reglas**: una opción no habilitada nunca debe ejecutarse (CA02) y una entrada inválida no debe alterar la partida (N03). (2) **Modificabilidad del contenido**: la necesidad principal de la escritora es cambiar historias sin programar (N01). (3) **Claridad**

de errores: el escritor debe saber qué corregir (N06). La facilidad de uso se atiende de forma básica (números o IDs, ayuda `?`), y el rendimiento no se prioriza porque las historias son pequeñas.

Decisión de responsabilidad profesional. [COMPLETAR/confirmar] Declaramos el uso de Claude Code (herramienta de IA) para generar código, pruebas y borradores de documentos; ningún resultado se presenta sin haberlo verificado (ver §10). Además, el contenido de las historias es de autoría del colectivo: la historia oficial se copió sin modificar y la segunda historia es original. Solo usamos dependencias con licencias permisivas (PSF, MIT, BSD, Apache-2.0) documentadas en `docs/arquitectura.md` §5, y la evidencia de RNF no publica rutas locales.

3. Proceso y agilidad

Proceso elegido. [COMPLETAR/confirmar] Proceso **incremental** con prácticas ágiles ligeras: incrementos cortos (modelo → validador → sesión → CLI → pruebas), cada uno cerrado con commit y pruebas. **Alternativa comparada:** cascada. Se descartó porque el cambio anunciado para la hora 6 hacía inviable congelar requisitos y diseño; el enfoque incremental permitió etiquetar `v1-inicial` y absorber el cambio en un incremento acotado.

Actividades en esta entrega. *Especificación*: RF/RNF con criterios Dado/Cuando/Entonces y preguntas (`docs/requisitos.md`). *Desarrollo*: commits pequeños que citan RF o pruebas. *Validación*: 301 pruebas automatizadas, verificación de RNF y recorridos manuales para el video. *Evolución*: solicitud de cambio analizada en `docs/analisis_impacto_cambio.md` y entregada como `v2-cambio`.

Prácticas ágiles. Utilizadas: (1) **backlog priorizado y tablero** con responsables (`docs/backlog.md`, capturas en el anexo); (2) **desarrollo guiado por pruebas de aceptación**: los CA01–CA03 y el ejemplo del docente se convirtieron en pruebas antes de dar el ítem por terminado. **Descartada:** [COMPLETAR/confirmar] *programación en pareja*, porque el trabajo fue individual; la revisión se sustituyó por pruebas automatizadas y una segunda lectura de los requisitos (ver §4).

Definición de terminado. Ver `docs/backlog.md` (7 criterios: RF con aceptación, motor sin datos de historias, pruebas pasando, sin tracebacks, documentos coherentes, commit pequeño).

Aprendizaje y acción de mejora. [COMPLETAR: un aprendizaje real y la acción concreta de mejora. Sugerencia a confirmar o reemplazar: «Una mutación manual mostró que ninguna prueba cubría elegir en un final antes de deshacer; acción: para cada regla nueva, comprobar que alguna prueba falla si se rompe la regla».]

4. Requisitos

17 RF atómicos (RF01–RF16 iniciales, RF17 del cambio), con cobertura N01: RF01–RF02; N02: RF03–RF04; N03: RF05–RF06, RF16–RF17; N04: RF07, RF17; N05: RF08–RF09, RF16–RF17; N06: RF10–RF15. Cada uno tiene criterio Dado/Cuando/Entonces (RF17 con uno por regla de la solicitud).

RNF medibles. RNF-01 Modificabilidad: 0 líneas cambiadas en `motor/` para jugar una historia nueva y 100 % de sus finales alcanzables. RNF-02 Claridad de errores: 100 % de los 10 códigos con causa e ID/campo, salida 2, 0 tracebacks. RNF-03 Confiabilidad: 0 excepciones y 0 violaciones de invariantes en 1 000 secuencias × 50 acciones por historia.

Preguntas. P1 retroceso (acuerdo provisional, resuelto por el cambio), P2 formato (supuesto: JSON), P3 escena sin opciones habilitadas (supuesto: informar y permitir reiniciar). [COMPLETAR: respuestas recibidas, si las hubo.]

Defecto detectado en revisión. Al ser un trabajo individual no hubo revisión entre integrantes; se hizo una revisión propia en una segunda pasada. [COMPLETAR: resumen del defecto real y su corrección; ver `docs/requisitos.md` §6.]

5. Modelos

Ver Anexo A: contexto, casos de uso, clases del dominio, secuencia «elegir una opción» (incluye la instantánea de deshacer) y estados de una sesión (EnCurso, Bloqueada, Finalizada y carga rechazada). Los nombres de clases, métodos y códigos son los del código de `v2-cambio`.

6. Arquitectura

Capas: contenido JSON → `cargador` → `validador` → `modelo` ← `sesion` ← `cli` (diagrama en el Anexo B). Decisiones: **AD-1** validar todo antes de jugar (vs. validación perezosa) y **AD-2** deshacer con una única instantánea (vs. pila, Command o reproducir elecciones); contexto, alternativas y consecuencias en `docs/arquitectura.md` §2. La separación contenido/reglas/interfaz se demuestra con `el_faro.json`, agregada sin cambiar `motor/`. Diseño detallado: `Sesion` no tiene E/S y devuelve `ResultadoAccion`, lo que permite probarla sin simular el teclado y convierte a la consola en un adaptador.

7. Implementación y pruebas

Python 3.10, biblioteca estándar y pytest (MIT). `motor/` tiene 7 archivos (5 módulos de lógica, `__init__` y `__main__`); ejecución con `python -m motor <ruta>` y `--validar`.

Conjunto	Ejecuciones	Resultado
U1 condiciones	7	pasan
U2 elección válida	5	pasan
U3 entrada inválida	8	pasan
U4 validación (21 variantes, 10 códigos)	51	pasan
U5 reiniciar (incl. extendido)	8	pasan
U6 deshacer (nuevo)	12	pasan
I1 integración (incl. extendido y segunda historia)	13	pasan
Bloqueo	3	pasan
CLI	11	pasan
RNF-01 / RNF-02 / RNF-03	5 / 23 / 2	pasan
Coherencia de la documentación (matriz y diagrama de clases vs. código)	153	pasan
Total	**301**	**301 pasan, 0 fallan**

Fuente: `evidencias/resultados_pruebas.txt` (pytest 9.1.1, Python 3.10.11). RNF: `evidencias/rnf/resultado_rnf.txt` — RNF-01 CUMPLE (0 literales, 0 líneas de `motor/`, finales alcanzados 2/2 y 3/3); RNF-02 CUMPLE (10/10 códigos, 21/21 casos, 0 tracebacks); RNF-03 CUMPLE (2 × 50 000 acciones, 0 excepciones, 0 violaciones). [COMPLETAR: si vuelven a ejecutar las pruebas antes de entregar, actualizar estos números con `python scripts/ejecutar_pruebas.py`.]

8. Cambio e impacto

La solicitud de la hora 6 reemplazó el acuerdo «sin retroceso» por **deshacer la última elección válida**. Versión de partida `v1-inicial` (`10dfc3a`, 2026-10-15 05:52:29 +0700); versión final `v2-cambio` (`bef51ab`).

Impacto: RF17 nuevo y RF06/RF08/RF09/RF16 ampliados; modelos de clases, secuencia, estados y casos de uso; componentes `sesion.py` (+23/−1) y `cli.py` (+9/−1); pruebas U6 nuevas y U5/I1 extendidas; backlog B13–B17; trazabilidad RF17. Las 95 pruebas de la versión inicial siguen pasando sin cambios en sus aserciones. Detalle y riesgos en `docs/analisis_impacto_cambio.md`.

9. Aportes

Trabajo individual: todos los commits de `evidencias/historial_git.txt` (autor `jaimeutb`) corresponden al único autor, con apoyo de Claude Code declarado en §10.

Autor	Trabajo propio	Generado con IA y verificado
[COMPLETAR: nombre]	[COMPLETAR: lo que hizo personalmente, p. ej. redactar el prompt, revisar requisitos, tablero, video, decisiones de las preguntas, revisión de código]	código, pruebas y borradores (ver §10), verificados mediante [COMPLETAR]

10. Fuentes y herramientas

Fuente / herramienta	Uso	Licencia
Python 3.10 y su biblioteca estándar	lenguaje, `json`, `dataclasses`, `zipfile`, `subprocess`	PSF License
pytest 9.1.1 (y dependencias: pluggy, iniconfig, packaging, colorama, exceptiongroup, tomli, Pygments)	pruebas automatizadas	MIT / BSD / Apache-2.0
Mermaid (mermaid.js 11, cargado desde jsDelivr al generar el PDF)	dibujo de los diagramas	MIT
Python-Markdown 3.10 y Google Chrome/Edge sin interfaz	conversión de `informe.md` / `anexo.md` a PDF (`scripts/generar_pdfs.py`)	BSD-3-Clause / navegador del sistema
Claude Code (Anthropic), herramienta de IA	Generó a partir de un prompt del autor (`PROMPT_CLAUDE_CODE.md`): estructura y código del motor, segunda historia, pruebas, scripts de verificación y empaquetado, y borradores de estos documentos.	servicio de IA

Cómo verificamos los resultados de la IA. [COMPLETAR con detalles verídicos: qué revisaron línea a línea, qué ejecutaron, qué corrigieron o rechazaron.] Verificaciones reproducibles que ya constan en el repositorio: ejecución completa de pytest (148 pasan), `scripts/verificar_rnf.py`, recorridos manuales de los tres finales y de la segunda historia (video), y comparación del archivo oficial copiado con el original (idénticos byte a byte). El código del motor no reutiliza fragmentos de tutoriales externos. [COMPLETAR: otras fuentes consultadas, si las hubo.]